回顾：顺序表与链表

顺序表	链表
连续存储单元依次存储数据元素，数据的物理关系与逻辑关系一致	用物理上离散的存储单元附加指针，存储数据元素
访问与存储：支持随机访问(O(1))，某些操作时间复杂度较低	不支持随机访问，很多操作涉及遍历整个表(O(n))
插入/删除需移动大量数据	插入/删除无需移动其他数据元素
存储空间利用率不高，表长变化不够灵活	存储空间利用灵活，指针存储有一定开销，表长易变化
实现直接	操作容易出错

数据结构分类

- **数据结构三元素：逻辑结构、存储结构、运算**

数据结构	逻辑	存储	运算(操作、接口)
顺序表	线性	顺序	查找、排序等，访问无约束
链表	线性	链式	查找、排序等，访问无约束
栈	线性	顺序或链式	访问约束：后进先出
队列	线性	顺序或链式	访问约束：先进先出

第三章栈和队列

栈

队列

递归

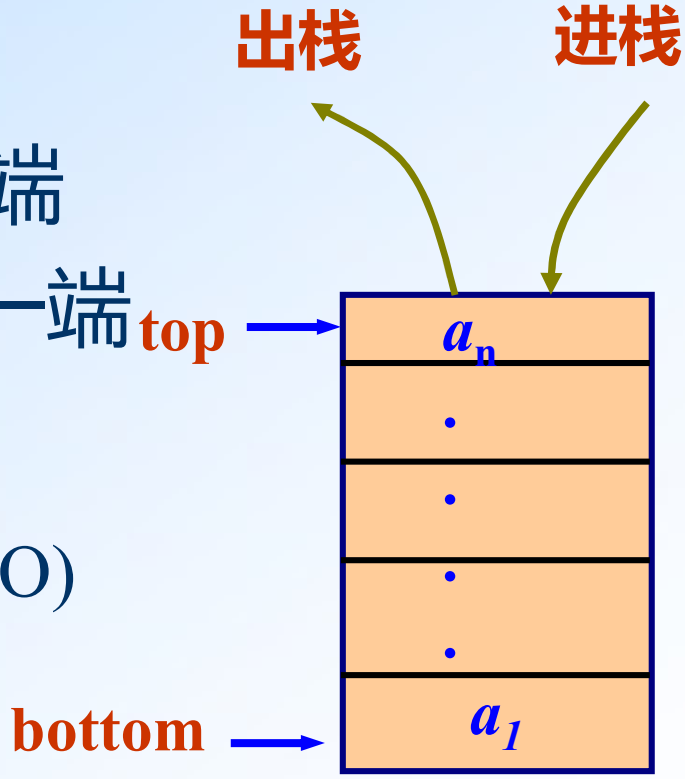
栈（Stack）

定义:是限定仅在表尾进行插入或删除操作的线性表。

允许插入和删除的一端

称为栈顶(top)，另一端称为栈底(bottom)

特点：后进先出 (LIFO)



栈

- 生活中的栈







只能从一端（顶端）进出，后进先出

栈的抽象数据类型（C语言）

栈的抽象数据类型定义为：

ADT Stack {

数据对象：$D=\{a_i|a_i \in ElemSet, i=1,2,...,n, n\geq 0\}$

数据关系：$R1=\{<a_{i-1}, a_i>|a_{i-1}, a_i \in D, i=2,...,n\}$

约定a_n端为栈顶,a_1端为栈底。

基本操作：

.....

} ADT Stack

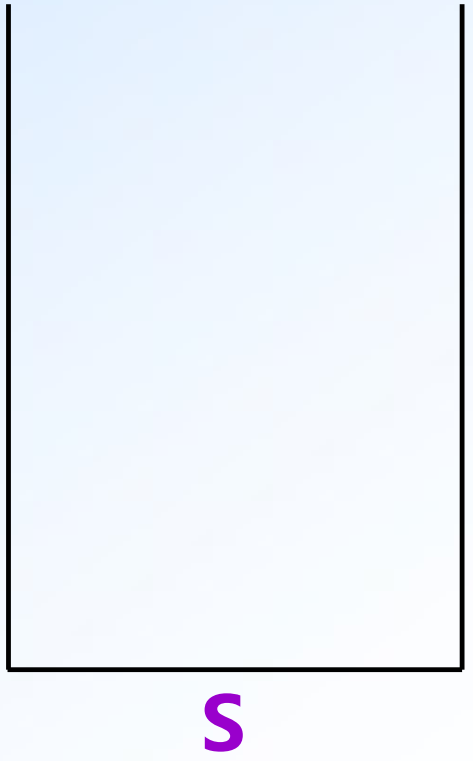
栈的抽象数据类型（C++）

```
template <class Type> class Stack {
public:
    Stack ( int sz = 10 );          //构造函数
    void Push ( Type& item);        //进栈
    Type Pop ( );                   //出栈
    Type GetTop ( );                //取栈顶元素
    void MakeEmpty ( );             //置空栈
    int IsEmpty ( ) const;          //判栈空否
    int IsFull ( ) const;           //判栈满否
}
```

栈

- 栈：只能在表的顶端进出（访问受限）的线性表
- 支持的操作（接口、运算）
  - ✓ 插入 push(x)
  - ✓ 删除（取出）pop()
  - ✓ 访问 top()
  - ✓ 判空 empty()
  - ✓ 求长 size()

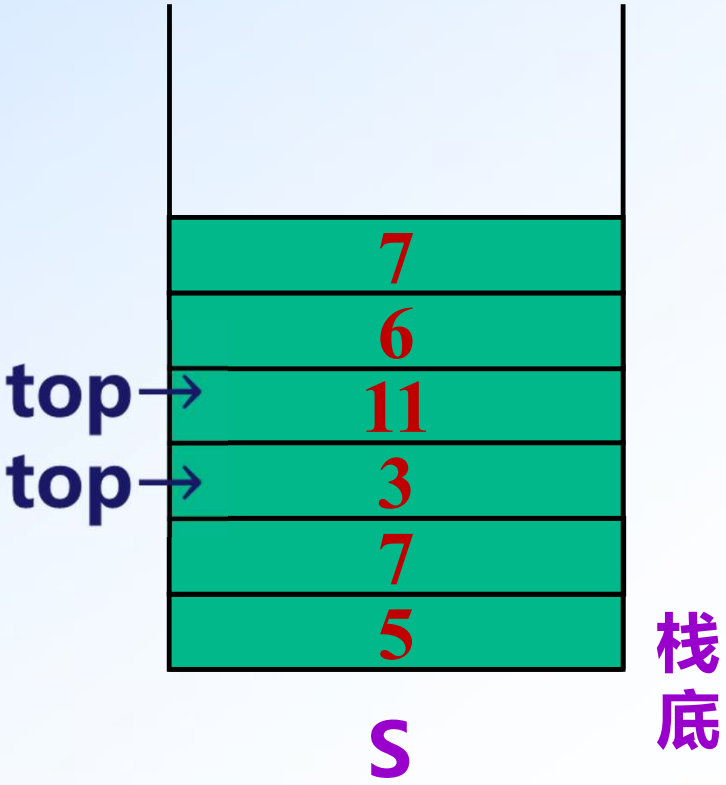
复杂度 O(1)



栈的基本操作

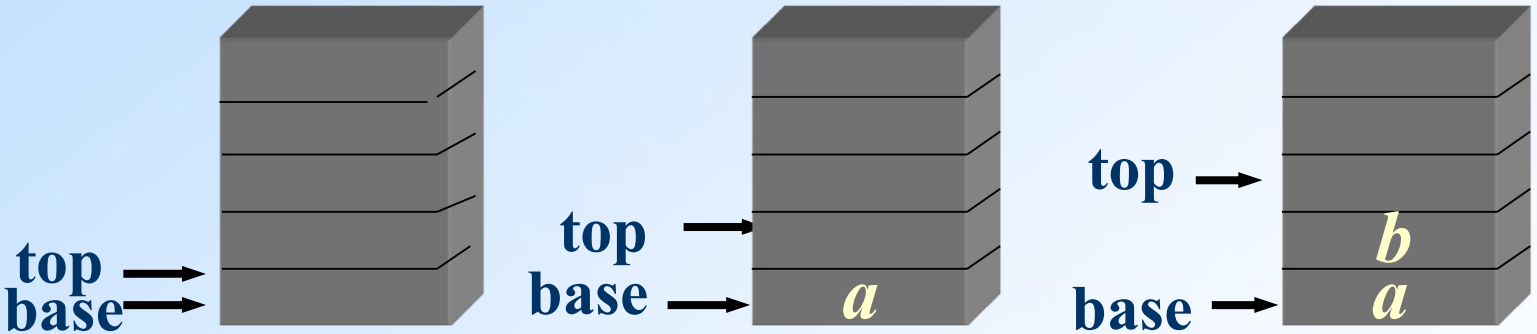
- 栈：只能在表的顶端进出（访问受限）的线性表

操作	输出	操作	输出
push(5)		size()	4
push(3)		push(6)	
pop()	3	empty()	false
push(7)		push(7)	
push(3)		pop()	7
top()	3	pop()	6
empty()	false	top()	11
push(11)			

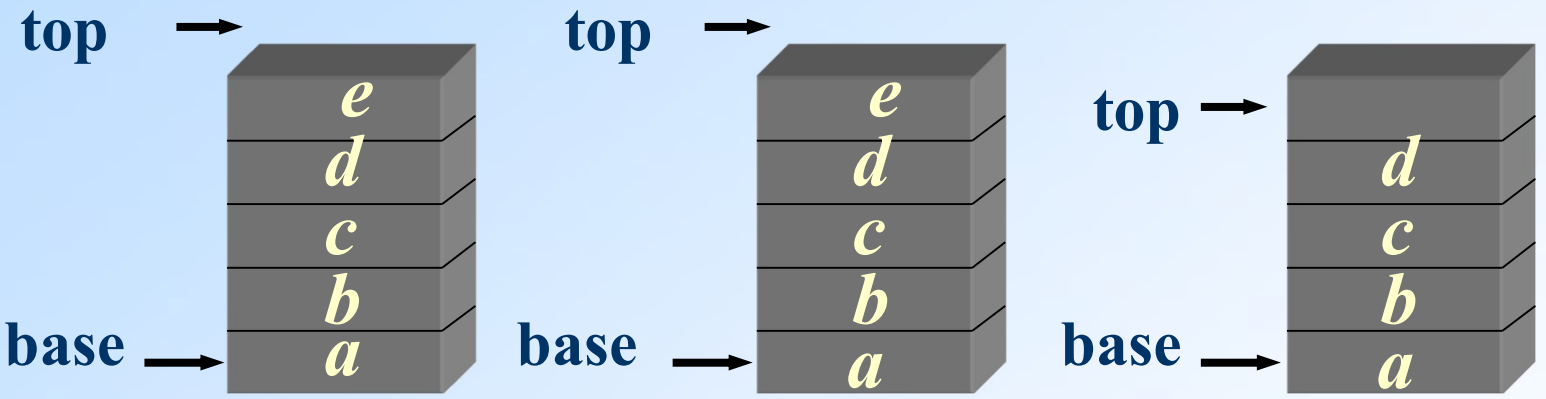


栈的表示和实现：**顺序栈和链栈**

顺序栈：栈的顺序存储结构，利用一组地址连续的存储单元依次存放自栈底到栈顶的数据元素，指针top指向栈顶元素在顺序栈中的下一个位置，base为栈底指针，指向栈底的位置。



空栈　　*a* 进栈　　*b* 进栈



e 进栈　　*f* 进栈溢出　　*e* 出栈

顺序栈的定义及存储表示

```
#define STACK_INIT_SIZE  100//栈存储空间的初始分配量
#define  STACKINCREMENT   10//栈存储空间的分配增量
typedef struct{
SElemType *base;//栈空间基址称为栈底指针，指向栈底位置
SElemType *top; //栈顶指针,约定栈顶指针指向栈顶元素的
                                //下（后面）一个位置;
int stacksize;      //当前分配的栈空间大小
                    // (以sizeof(SElemType)为单位)
}SqStack;// SqStack:: 结构类型名
```

初始化操作

参数：S是存放栈的结构变量

功能：建立一个空栈S

```
Status InitStack_Sq(SqStack &S)  {
        //构造一个空栈S
 S.base=(SElemType * )malloc(STACK_INIT_SIZE  *
      sizeof(SElemType));
      //为顺序栈动态分配存储空间
      if (! S. base) exit(OVERFLOW);     //存储分配失败
      S.top=S.base;
      S.stacksize=STACK_INIT_SIZE;
      return OK;                    }//InitStack_Sq
```

取栈顶操作

功能：取栈顶元素，并用e返回

```
Status GetTop_Sq(SqStack S, SelemType &e) {
        // 若栈不空，则用e返回S的栈顶元素，并返回OK；
       //否则返回ERROR
        if (S.top= =S.base)  return ERROR; //若栈为空
        e = * (S.top-1);
        return OK;
}//GetTop_Sq
```

```
int x = 10;
int* p = &x;
int& r = *p; // 通过指针创建引用
int& r2 = x; //引用是变量的别名，必须在初始化时绑定到一个变量
int* p2 = &r2; // 通过引用获取指针
```

进栈操作Push

功能：元素e进栈

```
Status Push(SqStack &S, SElemType e) {
    //将元素e插入栈成为新的栈顶元素,要考虑上溢情况
    if (S.top-S.base>=S.stacksize) {
        //栈满，追加存储空间
        S.base= (SElemType * )realloc(S.base,
      (S.stacksize +STACKINCREMENT) sizeof(ElemType));
        if (! S.base) exit(OVERFLOW);  //存储分配失败
        S.top=S.base+S.stacksize;
        S.stacksize+=STACKINCREMENT;  }
    *S.top++=e;      //元素e 插入栈顶，修改栈顶指针
    return OK;     }//Push
```

出栈操作Pop

功能：栈顶元素退栈，并用e返回

```
Status Pop(SqStack &S, SElemType &e) {
  //若栈不空，则删除S的栈顶元素，用e 返回其值并返回OK
//否则返回ERROR
      if (S.top= = S.base) return ERROR;
      //栈空，返回ERROR
      e = * --S.top; //优先级 '.' > "--" = '*'
      //删除栈顶元素，用e 返回其值，并修改栈顶指针
      return OK;
} //Pop
```

栈的数组表示 — 顺序栈（C++）

```
#include <assert.h>
template <class Type> class Stack {
private:
    int top;                        //栈顶指针
    Type *elements;                 //栈元素数组
    int maxSize;                    //栈最大容量
public:
    Stack ( int sz = 10 );          //构造函数
    ~Stack ( ) { delete [ ] elements; }
    void Push ( Type & item );      //进栈
```

```
    Type Pop ( );                          //出栈
    Type GetTop ( );                       //取栈顶
    void MakeEmpty ( ) { top = -1; }   //置空栈
    int IsEmpty ( ) const { return top == -1; }
    int IsFull ( ) const
        { return top == maxSize-1; }
}

template <class Type> Stack<Type> ::
Stack ( int s ) : top (-1), maxSize (s) {
    elements = new Type[maxSize];
    assert ( elements != NULL );        //断言
}
```

```
template <class Type> void Stack<Type> ::
Push ( Type & item ) {
    assert ( !IsFull ( ) );          //先决条件断言
    elements[++top] = item;          //加入新元素
}

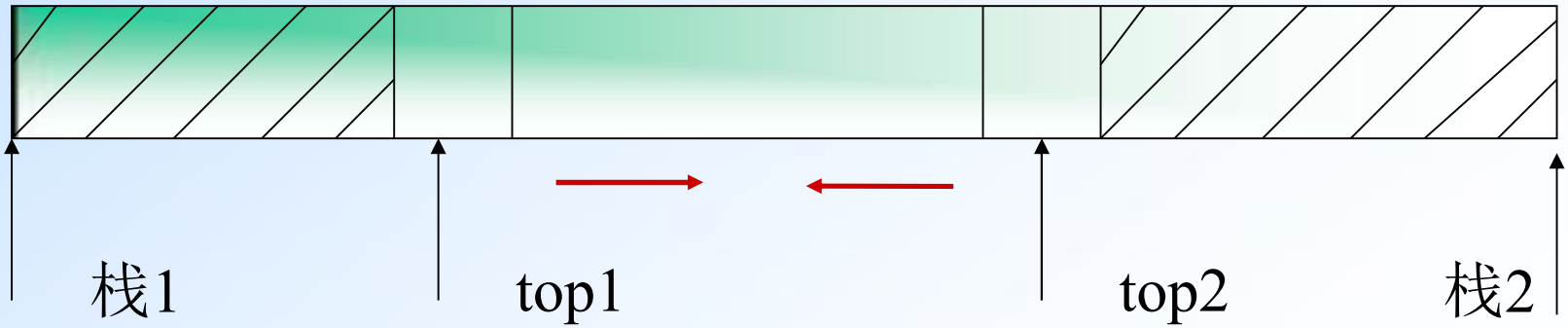
template <class Type> int stack<Type> ::
Pop (Type & item ) {
    if ( IsEmpty ( ) ) return 0;  //栈空不退栈
    item  =  elements[top];
    top--;   return 1;               //退出栈顶元素
}
```

```
template <class Type> Type stack<Type>::
GetTop ( ) {
    assert ( !IsEmpty ( ) );        //先决条件断言
    return elements[top];           //取出栈顶元素
}
```

程序中定义多个栈

（1）定义共享栈数据结构

```
#define MAX 100
  int stack[MAX],top1=0,top2=MAX-1;
```



(2)共享进栈算法

```
void push1(int x)
 {if (top1>top2)
  printf(“overflow”);
  else
   {stack[top1]=x; top1++;}}

void push2(int x)
 {if(top1>top2)
     printf(“overflow”);
  else {stack[top2]=x; top2--;}
```

(3)共享出栈算法

```
  int pop1()
{if top1==0){printf(“underflow”);return(NULL);}
top1--;return(stack[top1]);}

int pop2()
{if(top2==MAX-1)
{printf(“underflow”);return(NULL):}
   top2++;return(stack[top2]);}
```

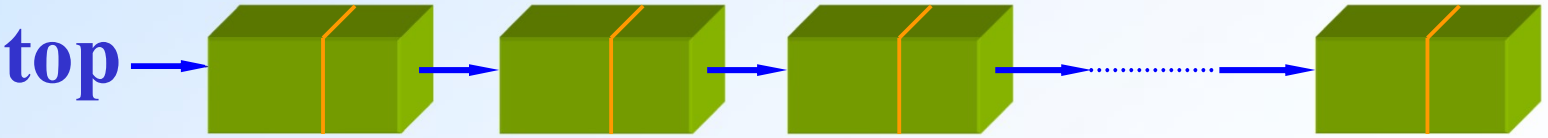
链式栈:栈的链接表示

链式栈无栈满问题，空间可扩充

插入与删除仅在栈顶处执行

链式栈的栈顶在链头

适合于多栈操作



链式栈 (LinkedStack)中结点的定义

```
typedef   struct LNode
{   ElemType data;
    struct  LNode  *next;
}* SLink;
```

链式栈 (LinkedStack)结构定义

```
typedef   struct
{   SLink  top;    //栈顶指针
    int  length;   //栈中元素个数
}LStack;
```

链式栈 (LinkedStack)的基本操作

```
void InitStack ( LStack &S )
    {
       // 构造一个空栈 S
       S.top = NULL;     // 设栈顶指针的初值为空
       S.length = 0;     // 空栈中元素个数为0
    } // InitStack
Status GetTop(LStack &S, ElemType &e) {
       // 若栈空则返回ERROR ,否则返回OK
       if (S.top= =NULL)  return ERROR; //若栈为空
       e = S.top->data;
       return OK;
}//GetTop
```

点号（.）：左边必须为实体
箭头（->）：左边必须为指针

链式栈 (LinkedStack)的基本操作

```
void Push ( LStack &S, ElemType e )
  {
    // 在栈顶之上插入元素 e 为新的栈顶元素
    p = (LNode *)malloc(sizeof(LNode);
                              // 建新的结点
    if(!p) exit(1);          // 存储分配失败
    p -> data = e;
    p -> next = S.top;     // 链接到原来的栈顶
    S.top = p;               // 移动栈顶指针
    ++S.length;                 // 栈的长度增1
  } // Push
```

链式栈 (LinkedStack)的基本操作

```
bool Pop ( LStack &S, ElemType &e )
  {
    // 若栈不空，则删除S的栈顶元素，用 e 返回其值，
    // 并返回 TRUE；否则返回 FALSE
    if ( !S.top )
       return FALSE;
    else
    {
     e = S.top -> data;          // 返回栈顶元素
     q = S.top;
     S.top = S.top -> next;      // 修改栈顶指针
     delete q;                   // 释放被删除的结点空间
     --S.length;                 // 栈的长度减1
     return TRUE;
    }
  } // Pop
```

链式栈 (LinkedStack)类的定义（C++）

```
template <class Type> class Stack;

template <class Type> class StackNode {
friend class Stack<Type>;
private:
    Type data;                          //结点数据
    StackNode<Type> *link;              //结点链指针
public:
    StackNode ( Type d = 0, StackNode<Type>
        *l = NULL ) : data ( d ), link ( l ) { }
};
```

```
template <class Type> class Stack {
private:
    StackNode<Type> *top;        //栈顶指针
public:
    Stack ( ) : top ( NULL ) { }
    ~Stack ( );
    void Push ( const Type & item);   //进栈
    int Pop ( );                      //退栈
    Type *GetTop ( );         //读取栈顶元素
    void MakeEmpty ( );   //实现与~Stack( )同
    int IsEmpty ( ) { return top == NULL; }
}
```

```
template <class Type> Stack<Type> ::
~Stack ( ) {
    StackNode<Type> *p;
    while ( top != NULL )        //逐结点回收
        { p = top;  top = top->link;  delete p; }
}

template <class Type> void Stack<Type> ::
Push ( const Type &item ) {
   top = new StackNode<Type> ( item, top );
   //新结点链入*top之前，并成为新栈顶
}
```

```
template <class Type> int Stack<Type> ::
Pop ( ) {
    if ( IsEmpty ( ) ) return 0;
    StackNode<Type> *p = top;
    top = top->link;                    //修改栈顶指针
    delete p;   return 1;               //释放
}

template <class Type> Type Stack<Type>::
GetTop ( ) {
    assert ( !IsEmpty ( ) );
    return top->data;
}
```



出栈顺序

设有编号为1，2，3，4的四辆列车，顺序进入一个栈式结构的车站，具体写出这四辆列车开出车站的所有可能的顺序

① 全进之后再出情况，只有1种：4，3，2，1

② 进3个之后再出的情况，有3种，3,4,2,1 3,2,4,1 3,2,1,4

③ 进2个之后再出的情况，有5种，2,4,3,1 2,3,4,1 2,1, 3,4 2,1,4,3 2,3,1,4

④ 进1个之后再出的情况，有5种，1,4,3,2 1,3,2,4 1,3,4,2 1, 2,3,4 1,2,4,3

出栈的每个元素后面比该元素小的元素，应该按从大到小的相对顺序排列（逆序）

① 全进之后再出情况，只有1种：4，3，2，1

② 进3个之后再出的情况，有3种，3,4,2,1 3,2,4,1 3,2,1,4

③ 进2个之后再出的情况，有5种，2,4,3,1 2,3,4,1 2,1, 3,4 2,1,4,3 2,3,1,4

④ 进1个之后再出的情况，有5种，1,4,3,2 1,3,2,4 1,3,4,2 1, 2,3,4 1,2,4,3

1. 顺序进栈，再顺序出栈；
2. 第一个出来的是3，剩下的12必须是逆序出来，4可以在12的3个位置进出栈，一共3种可能；
3. 第一个出来的是2，剩下1，未进栈的34可以一起进出栈的位置有2个，每一个位置又有两种顺序情况，还有一个被1劈开进出栈的情况，一共5种可能；
4. 第一个出来的是1，后面的分：3个一起进出（1种）、1和2固定后分3和4一起在两个位置（含43, 3种）、3和4被2分开的只有1种。

任何出栈的元素后面出栈的元素必须满足以下三点：

1、在原序列中相对位置比它小的，必须是逆序；
2、在原序列中相对位置比它大的，顺序没有要求；
3、以上两点可以间插进行。